SY B.Tech. – ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PRACTICAL ASSIGNMENT TEMPLATE

1. Student Details

Student Name	Pranav Chopade
PRN	202501110025
Division	A
Batch	A1
Roll No.	202501110025
Date of Submission	23-10-2026

2. Practical Details

Practical No.	8
Practical Title	IMPLEMENT EXCEPTION HANDLING OPERATIONS WITHOUT stdexcept
Date Performed	23-10-2026
Faculty Name	Dr. Khushal Khairnar

3. Problem Statement

Write a C++ program to demonstrate exception handling using try, throw, and catch without including the **<stdexcept>** library. Handle division by zero by throwing a string literal, handle a negative numerator by throwing an integer, and include a catch-all handler for other exceptions.

4. Objectives

1. Understand exception handling in C++.

2. Use try, throw, and catch without including stdexcept.

3. Demonstrate handling of different exception types using const char* and int.

4. Include a catch-all handler for otherwise unmatched exceptions.

5. Expected Learning Outcomes

After completing this practical, the student will be able to:

1. Identify exceptional conditions in a program.

2. Throw and catch exceptions of different types.

3. Explain the purpose of multiple catch blocks and catch(...).

4. Implement basic exception handling using only the iostream library.

6. Theory / Concept

try: Contains statements that may throw an exception.

throw: Signals an exceptional condition and transfers control to a matching handler.

catch: Handles an exception thrown from the try block.

Typed catch: A catch block can handle a specific type, such as const char* or int.

Catch-all: catch(...) handles an exception that does not match an earlier handler.

Note: This program uses only <iostream>; it does not include <stdexcept>.

7. Algorithm / Pseudocode

START

1. Read numerator a and denominator b.
2. Enter the try block.
3. If b equals zero, throw a string literal describing the error.
4. If a is negative, throw the integer value a.
5. Otherwise, display the division result.
6. Catch const char* and display the error message.
7. Catch int and display the negative-numerator message.
8. Use catch(...) for any other exception type.
9. Display a message showing execution continues after handling.

STOP

8. Technology / Tools / Libraries Used

Programming Language	C++
IDE / Platform	Dev C++
Libraries / Packages	iostream only; no stdexcept

9. Implementation / GitHub Repository

GitHub Repository URL	https://github.com/pranav-4797/OOP-Practical-Assignment
Repository Name	OOP-Practical-Assignment
Practical Folder	Practical 8
Branch Used	main
Commit ID / Latest Commit	Not provided

Repository Contents Used

The source file is provided separately as “Practical_8_Exception_Handling.cpp”. Add it to the Practical 8 folder of the repository if required.

10. Test Cases / Input and Output

Test Case	Input	Expected Output	Status
1	a = 10, b = 0	String exception: Division by zero is not allowed.	Pass (sample)
2	a = -5, b = 2	Integer exception for negative numerator	Pass (sample)
3	a = 10, b = 2	Result: 5	Pass (logic)

11. Results / Observations

The program demonstrates try, throw, and catch without using the stdexcept library. A string literal is thrown for division by zero, an integer is thrown for a negative numerator, and catch(...) provides a fallback handler.

12. Source Code

```
#include <iostream>
using namespace std;

int main() {
int a, b;

cout << "Enter numerator: ";
cin >> a;
cout << "Enter denominator: ";
cin >> b;

try {
if (b == 0) {
throw "Division by zero is not allowed.";
}

if (a < 0) {
```

```
throw a;
}

cout << "Result: " << a / b << endl;
}
catch (const char* message) {
cout << "String exception: " << message << endl;
}
catch (int value) {
cout << "Integer exception: numerator cannot be negative ("
<< value << ")." << endl;
}
catch (...) {
cout << "Unknown exception occurred." << endl;
}

cout << "Program continues after exception handling." << endl;
return 0;
}
```

Screenshots / Output

```
Practical 8 - Exception Handling - Output

Run 1: Division by zero
Enter numerator: 10
Enter denominator: 0
String exception: Division by zero is not allowed.
Program continues after exception handling.

Run 2: Negative numerator
Enter numerator: -5
Enter denominator: 2
Integer exception: numerator cannot be negative (-5).
Program continues after exception handling.
```

This is a representative terminal-style output image for the two sample exception cases. Run the program in Dev-C++ to capture an actual IDE screenshot if required.

13. Conclusion

This practical demonstrates C++ exception handling using try, throw, and catch without including stdexcept. Typed handlers process const char* and integer exceptions, and the catch-all handler is included for other exception types.

14. Assignment Questions

Q1. What is exception handling?
Answer: A mechanism for detecting and handling exceptional conditions during program execution.

Q2. What is the purpose of throw?
Answer: It raises an exception that can be handled by a matching catch block.

Q3. Can exceptions be used without stdexcept?
Answer: Yes. C++ allows throwing and catching built-in types and other values without including stdexcept.

Q4. What does catch(...) do?
Answer: It catches exceptions that do not match preceding catch handlers.

Q5. Which library is used in this program?
Answer: Only iostream is included.

15. Student Declaration

I hereby declare that the above practical work has been completed by me and that the implementation, analysis, and answers are my own work.

Student Name	Pranav Chopade
PRN	202501110025
Signature	